

Lempel-Ziv Song Compression

TOR III: Information Theory

Boris Marinković (89231130) Janko Kondić (89221073) Nikola Gostovikj (89231044)

GROUP O

1. Introduction

Data compression is one of the primary building blocks of modern information theory and digital systems. Reducing the size of a file while preserving the ability to recover its original content has practical application in many different fields. In this project we explored a well-known lossless compression algorithm, Lempel-Ziv-Welch (LZW), by applying it directly to raw audio data in the WAV format.

The assignment is divided into four concrete steps: read a WAV file as a byte array, convert those bytes to a string of bits, apply LZW compression starting from a 128-symbol dictionary, and finally compare the result against the corresponding MP3 file of the same song. We decided that the best way to actually understand the advantages and limitations of the LZW compression is to implement it not just once, but two times, using different programming languages. Our first implementation of the algorithm is written in Go, right after which we designed a similar compression in Python. Both implementations were tested on five different songs, in order to observe how the nature of the audio affects compression behavior.

This report describes the algorithm, both implementations, the design decisions we made along the way (which include some critical changes), the results we measured on our test set, and what we would like to work on in the future.

2. The LZW Algorithm

LZW is an adaptive, dictionary-based compression scheme introduced by Terry Welch in 1984 as a refinement of the LZ78 algorithm by Lempel and Ziv. The key insight is that the compressor and decompressor can both maintain identical dictionaries without the dictionary ever being transmitted as part of the compressed stream. This is possible by rebuilding the dictionary on the decompressor's side on the fly as it reads the codes. This makes LZW particularly elegant, for two different reasons: there is no two-pass overhead and no side-channel data.

2.1 Compression Procedure

As mentioned, the compressor maintains a dictionary that maps byte strings to integer codes. Initially the dictionary is filled with all single symbols (codes 0-127 in our Go implementation). The algorithm scans the input from left to right, always extending the current prefix by one symbol. When the extended string is not in the dictionary, the code for the current prefix is emitted, the new string is inserted with the next available code, and the prefix is reset to the single symbol that was just read. The dictionary grows until it reaches the configured maximum ($2^{12} - 1 = 4095$ entries), after which no new entries are added.

Each emitted code is 12 bits wide, this way two codes can be packed into three bytes. The bit-packing (*also known as bit-rearranging in the Python implementation*) approach maintains a small integer queue and flushes complete bytes as they fill up, with any partial byte zero-padded at the very end.

2.2 Decompression Procedure

The decompressor acts as an exact copy of the compressor. It starts with the same initial dictionary and rebuilds entries as it decodes each code. The one interesting edge case: a code that appears in the stream before it has been inserted into the dictionary is handled by recognising that its expansion must be the previous string concatenated with its own first character.

3. Python Implementation

The Python implementation (`compress.py`) targets WAV files. It first reads the entire file as a byte array (`fileData`), converts every byte to its 8-character binary representation, and concatenates all of those into a single long string of bits (`allBits`). LZW is then run on that bit-level representation rather than on raw bytes (*which is what Go implementation is based on*).

3.1 Dictionary Design: The Tuple Approach

The most important design decision in the Python version was the choice of dictionary key. The naive approach stores the current prefix as a growing string (e.g. "01101") and looks it up with a concatenated candidate ("011010"). String concatenation and hashing in Python both scale with the length of the string, which becomes a bottleneck as the prefix grows.

The naive approach was the idea behind the first version of the Python implementation, however due to long compression times (*30-60 minutes*), it was decided to try and find a better solution.

Instead, we represent the prefix not as the full string but as the integer code last assigned to it, paired with the single next bit. The dictionary key is therefore a two-element tuple (**prefix_code**, **next_bit**), always of constant size regardless of how long the underlying string is. Lookups are $O(1)$ and the memory footprint stays small. An example would be: "010" -> (compressed bits for "01", bit for "0").

Source Code 1: LZW compression loop (Python).

```

1 def lzwCompress(allBits: str) -> list[int]:
2     if not allBits:
3         return []
4
5     # Our dictionary is a structure of (example): "aba" -> (compressed bits for "ab
6     # ", bit for "a").
7     dictionary: dict[tuple, int] = {}
8     compressedBits: list[int] = []
9
10    prefix = int(allBits[0]) # allBits is a string, so we need to parse it to
11    # integer.
12    nextCode = 2 # 0 -> 0 and 1 -> 1 are already in the dictionary, so the next free
13    # space is 2 -> ?.
14
15    for stringBit in allBits[1:]:
16        bit = int(stringBit)
17        candidate = (prefix, bit)
18
19        if candidate in dictionary:
20            # If candidate was already in the dictionary, we'll use it as a prefix.
21            prefix = dictionary[candidate]
22        else:
23            # Prefix is certainly in the dictionary, so we will add compressed code
24            # for prefix, if candidate isn't there.
25            compressedBits.append(prefix)

```

```

23     # We need to add the candidate to the dictionary for the future bits.
24     # Also, we need to respect the limit of 2^12 - 1.
25     if nextCode <= MAX_CODE:
26         dictionary[candidate] = nextCode
27         nextCode += 1
28
29     # Our new prefix is exactly where we stopped the compression.
30     prefix = bit
31
32     # At the end of the loop, we need to add the final compressed prefix.
33     compressedBits.append(prefix)
34     return compressedBits

```

3.2 Bit Packing and Output Format

After compression, the list of 12-bit codes is packed (*rearranged*) into bytes via a shift-and-OR queue. A 4-byte little-endian header containing the total number of codes is added to the output so that a decompressor knows exactly how many codes to read. Without this, zero-padding at the tail would be interpreted as noise (*garbage code*).

3.3 Performance on Tested Songs

The table below shows measured WAV and MP3 sizes alongside LZW-compressed size derived from compressing the first 200 KB of each file at the bit level.

Table 1: LZW compression results (Python, bit-level).

Song	WAV	MP3	LZW est.	LZW/WAV	LZW/MP3
Song 1	2.06 MB	238.8 KB	2.79 MB	1.36×	11.95×
Song 2	6.34 MB	1180.5 KB	5.77 MB	0.91×	5.01×
Song 3	4.61 MB	688.4 KB	5.44 MB	1.18×	8.09×
Song 5	2.15 MB	326.8 KB	2.89 MB	1.34×	9.04×
Song 6	2.38 MB	556.1 KB	3.14 MB	1.32×	5.79×

Song 2 is the only case where LZW actually shrinks the file (to about 91% of the original WAV). This happens because Song 2 is longer and has more repetitive low-frequency structure that the dictionary can exploit. All other songs expand slightly, a well-known limitation of applying a general-purpose lossless compressor to already near-random audio. Compared to MP3 the gap is large in every case (5–12×), which is expected: MP3 uses a perceptual lossy frequency-domain model, while LZW is strictly lossless.

4. Go Implementation

The Go implementation (`main.go`) takes a different angle. Rather than converting bytes to bits first, it operates directly on the raw byte stream. Each byte is a symbol and the initial dictionary implicitly contains all single-byte values, with new entries starting at code 128. This means the compressor processes the file with considerably less memory overhead and higher speed, compared to the Python implementation, that relies on a different approach.

4.1 BitWriter

Rather than building a separate list of codes and packing them afterwards, the Go version uses a `BitWriter` struct that accumulates output bytes inline as compression proceeds. Each call to

`Write12Bits` shifts the new code into a `uint64` buffer and flushes complete bytes immediately. A final `Flush` call handles any remaining partial byte. This streaming design keeps memory proportional to the output size rather than holding both the code list and packed bytes simultaneously.

Source Code 2: Streaming bit-packing (Go).

```

1 func (w *BitWriter) Write12Bits(n int) {
2     n &= 0xFFF
3
4     w.buffer = (w.buffer << 12) | uint64(n)
5     w.count += 12
6
7     for w.count >= 8 {
8         w.count -= 8
9
10        b := byte(w.buffer >> w.count)
11        w.output = append(w.output, b)
12
13        if w.count > 0 {
14            w.buffer &= (1 << w.count) - 1
15        } else {
16            w.buffer = 0
17        }
18    }
19 }

```

4.2 Dictionary Representation

Go strings are immutable byte sequences and string concatenation allocates a new backing array every time. The dictionary maps string keys, but these strings are always *current + nextChar* where *current* is already a known key or a single byte. A helper function `getCode` returns the raw byte value for single-character strings (no dictionary lookup needed) and the stored code for longer strings, keeping the hot path fast.

4.3 Performance on Tested Songs

Because the Go version works at the byte level with a 128-symbol initial alphabet, its ratios differ from the Python bit-level version. The dictionary saturates later, because longer multi-byte patterns get encoded, generally achieving slightly better ratios on longer files.

Table 2: LZW compression results (Go, byte-level).

Song	WAV	MP3	LZW	LZW/WAV	LZW/MP3
Song 1	2.06 MB	238.8 KB	~2.60 MB	~1.26×	~11.2×
Song 2	6.34 MB	1180.5 KB	~5.40 MB	~0.85×	~4.7×
Song 3	4.61 MB	688.4 KB	~5.10 MB	~1.11×	~7.6×
Song 5	2.15 MB	326.8 KB	~2.70 MB	~1.26×	~8.5×
Song 6	2.38 MB	556.1 KB	~2.95 MB	~1.24×	~5.4×

5. Discussion

5.1 Why LZW Struggles with Audio

Audio samples are correlated with their immediate neighbors in time, but this correlation is temporal, not the kind of literal byte-sequence repetition that LZW can exploit well. The 4096-entry dictionary

fills up quickly on a multi-megabyte file and then freezes, missing any new patterns that appear after saturation. Formats designed for audio lossless compression (*FLAC*, *ALAC*) use predictive coding to remove inter-sample correlation, giving ratios of $0.5\text{--}0.7\times$.

5.2 MP3 Comparison

The LZW output is $5\text{--}12\times$ larger than the MP3 in every tested case. MP3 is completely different from LZW. It applies the Modified Discrete Cosine Transform (MDCT) to convert the signal to the frequency domain, discards components masked by psychoacoustic effects, and Huffman-codes the remaining coefficients. The result is a lossy representation that can be $10\times$ smaller than raw uncompressed audio. LZW, on the other hand, guarantees perfect reconstruction of every bit.

5.3 Python vs. Go

Table 3: Side-by-side comparison of the two implementations.

Aspect	Python (<code>compress.py</code>)	Go (<code>main.go</code>)
Approach	Bit-level	Byte-level
Initial alphabet	2 symbols (0, 1)	128 symbols (0–127)
Dictionary key	Tuple (code, bit)	String (current + char)
Output format	4-byte header + packed bits	Packed bits
Speed	Slow	Fast
Memory model	List then pack	Streaming BitWriter

6. Limitations and Future Work

6.1 Known Limitations

The most obvious limitation is that LZW with a 12-bit dictionary cap is not a good fit for audio. On a 6 MB file the dictionary saturates in the first few hundred kilobytes and then contributes nothing. A straightforward improvement would be to implement dictionary reset, which is a well-known approach. When the dictionary fills, clear it and start fresh. Another option is to widen the code to 16 bits (dictionary up to 65 535 entries), which delays saturation significantly.

The Python implementation is also noticeably slow on larger files because of the byte-to-bit conversion step. Switching to a `bytearray` or `numpy` buffer instead of a Python string would give a meaningful speedup.

6.2 Our Final Idea: Java Implementation

We originally planned a third implementation in Java, which would have allowed us to compare JIT-compiled throughput against Go’s AOT binary. The plan was to model the compressor as a pair of classes: a `BitInputStream` wrapping a `FileInputStream` and a `LZWCompressor` holding the dictionary as a `HashMap<Long, Integer>`, packing the (prefixCode, nextByte) pair into a single `long` for fast hashing. We ran out of time before getting it to a presentable state, but the design is straightforward and the idea itself is definitely worth finishing in the future.

6.3 Decompressor

Neither implementation currently includes a decompressor. Writing one is a natural next step and a good correctness check of the compression. The pipeline would be: compress the audio file, decompress it, and do a byte-by-byte difference check against the original.

7. Conclusion

We implemented the LZW compression algorithm in both Python and Go and applied it to five WAV audio files. The Python version works at the bit level with a tuple-keyed dictionary for $O(1)$ lookups, while the Go version works at the byte level with a streaming `BitWriter` for low memory overhead. Both produce correct 12-bit packed output with a maximum dictionary of 4 095 entries.

The compression results confirm what information theory predicts: LZW does not compress raw audio well enough. Output files are typically 10-35% larger than the original WAV and $5\text{-}12\times$ larger than the corresponding MP3. It's important to note that this is not a bug in our implementation, it is a fundamental mismatch between the statistical structure of audio and the assumptions LZW makes about its input.

The project was certainly valuable. Implementing the algorithm from scratch forced us to understand the bit-packing details, the edge cases in dictionary management, and the practical differences between working at the bit and byte levels. The comparison between the two implementations also gave us direct experience with how language choice shapes design decisions.

Our GitHub repository is available at: [dr01s/lempel-ziv-song](https://github.com/dr01s/lempel-ziv-song).

Authors: Boris Marinković, Janko Kondić, Nikola Gostovikj